

NumPy in Three Minutes

Just enough to read the code in the series · video 2:03

This lesson is a quick tour of **NumPy**, Python's library for fast math on arrays of numbers. It covers just enough to read every code snippet in the series: **shapes**, element-wise arithmetic, the **@** sign, **axis** and **keepdims**, **broadcasting**, **reshape** and **masks**. Most questions ask you to **predict** what a line of code gives. Write your prediction down first, then run the code to check.

How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 11).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

Concepts in this lesson

CONCEPT	VIDEO SEGMENT	QUESTIONS	MASTERED
1. Arrays and shapes	▶ 0:08-0:29	3	☐
2. Arithmetic on every element at once	▶ 0:29-0:39	3	☐
3. The @ sign: matrix multiplication	▶ 0:40-0:49	3	☐
4. Summing along an axis	▶ 0:50-1:01	3	☐
5. Broadcasting	▶ 1:02-1:15	4	☐
6. Reshape and transpose	▶ 1:16-1:25	4	☐
7. Masks, and it all together	▶ 1:26-1:46	4	☐

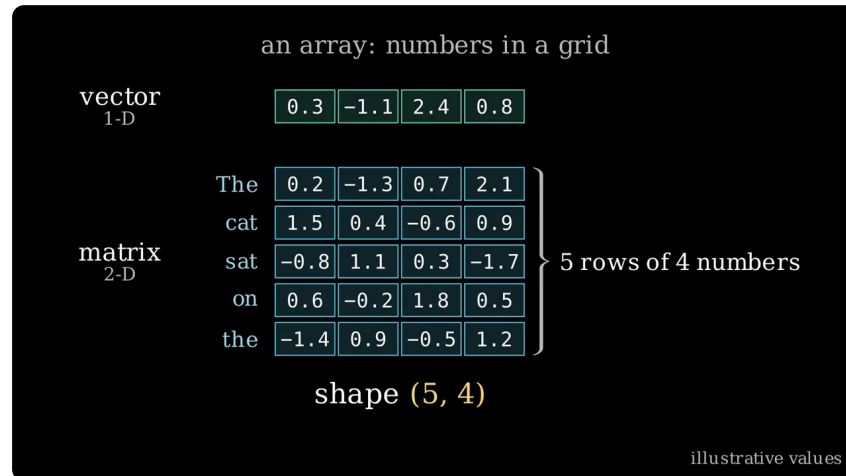
Before you start: this lesson puts earlier Foundations into code: vectors (F01), the dot product (F02), matrices (F03) and softmax (F07). You will meet this NumPy in the code of episodes 3, 6, 7, 9 and 13. To run the code questions you need Python with NumPy (`pip install numpy`).

CONCEPT 1 OF 7

Arrays and shapes

▶ Video 0:08-0:29

NumPy works with **arrays**: numbers in a grid. A **vector** is a one-dimensional (1-D) array, a **matrix** a two-dimensional (2-D) one. Every array has a **shape**: the matrix below, one row per token of “The cat sat on the”, has shape **(5, 4)**, 5 rows of 4 numbers. A 1-D shape has a trailing comma: a vector of 4 numbers has shape **(4,)**.



▶ 0:28 A vector (1-D), and a matrix (2-D) of shape (5, 4).

KEY IDEA

An array is a grid of numbers, and its **shape** lists the size of each dimension, rows first. In the series, a matrix of shape (tokens, numbers per token) is everywhere.

Check 1: Arrays and shapes

Answer all 3 questions before looking at the answers. Pass mark: all correct.

1.1 MULTIPLE CHOICE

An array holds 3 rows of 2 numbers each. What is its shape?

- A. (2, 3)
- B. (3, 2)
- C. (6,)
- D. (3,)

1.2 CALCULATION

A sentence has 7 tokens, and each token is a vector of 768 numbers. Stacked as one matrix (one row per token, as in the picture), what is its shape, and how many numbers does it hold?

1.3 HANDS-ON CODE

Predict the three shapes this prints. (X[0] is the first row.)

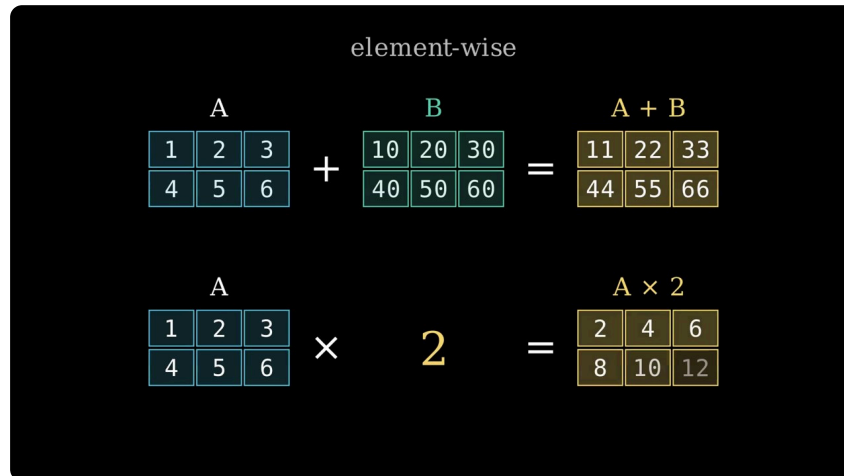
```
import numpy as np
v = np.array([0.3, -1.1, 2.4, 0.8])
X = np.zeros((5, 4))      # 5 rows of 4 zeros
print(v.shape, X.shape, X[0].shape)
```

CONCEPT 2 OF 7

Arithmetic on every element at once

▶ Video 0:29-0:39

Arithmetic works on **every element at once**, with no loops. Add two arrays of the **same shape** and matching elements are added: $1 + 10$, $2 + 20$, and so on. Multiply by 2 and every number doubles. The same goes for $-$, $/$ and $**$, and for $*$ between two arrays: $A * B$ multiplies matching elements.



▶ 0:38 A + B adds matching elements; A × 2 doubles every number.

KEY IDEA

Arithmetic is **element-wise**: each cell of the result uses only the matching cells of the inputs. A single number, like the 2 in $A \times 2$, applies to every element.

Check 2: Arithmetic on every element at once

Answer all 3 questions before looking at the answers. Pass mark: all correct.

2.1 HANDS-ON CODE

Work out the three printed arrays by hand, then run it.

```
import numpy as np
A = np.array([[1, 2, 3], [4, 5, 6]])
B = np.array([[10, 20, 30], [40, 50, 60]])
print(B - A)
print(A * 2 + 1)
print(A * B)
```

2.2 TRUE OR FALSE

"A of shape (2, 3) plus B of shape (3, 2) works, because both hold 6 numbers."

True ☐ False ☐

2.3 MULTIPLE CHOICE

`x = np.array([1.0, 2.0, 3.0])`. Which expression gives `[1, 4, 9]`?

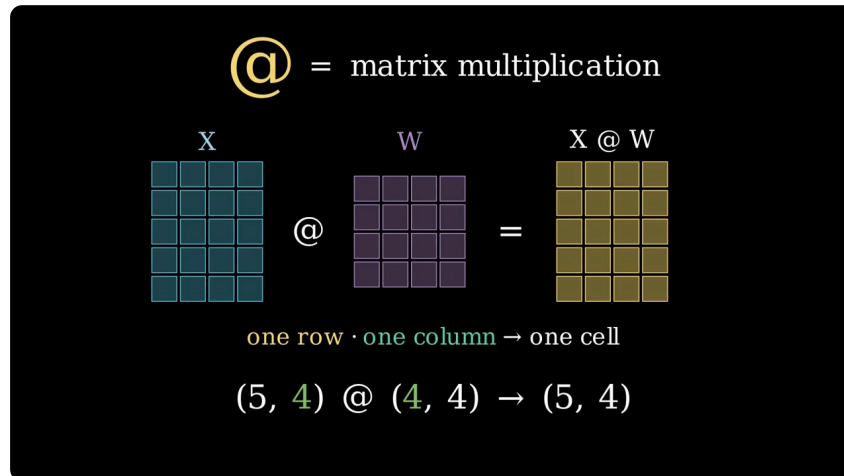
- A. `x ** 2`
- B. `x @ x`
- C. `x * 2`
- D. `np.sum(x) ** 2`

CONCEPT 3 OF 7

The @ sign: matrix multiplication

▶ Video 0:40-0:49

@ is **matrix multiplication**: each cell of the result is the **dot product** of one row of the left array with one column of the right array (F02, F03). For the shapes: **(5, 4) @ (4, 4) → (5, 4)**. The inner sizes (the green 4s) must match; the result takes its rows from the left array and its columns from the right one.



▶ 0:48 Each cell of X @ W: one row of X dotted with one column of W.

KEY IDEA

(n, k) @ (k, m) → (n, m), and the inner sizes must match. @ means dot products of rows with columns; * is element by element.

Check 3: The @ sign: matrix multiplication

Answer all 3 questions before looking at the answers. Pass mark: all correct.

3.1 CALCULATION

What is the shape of (a) (5, 4) @ (4, 8)? (b) (5, 64) @ (64, 5)? (c) (4, 5) @ (4, 4)?

3.2 TRUE OR FALSE

"In (5, 4) @ (4, 4), each of the 20 cells of the result is a dot product of 4 pairs of numbers."

True ☐ False ☐

3.3 HANDS-ON CODE

Predict both results by hand, then run it. Why are they different?

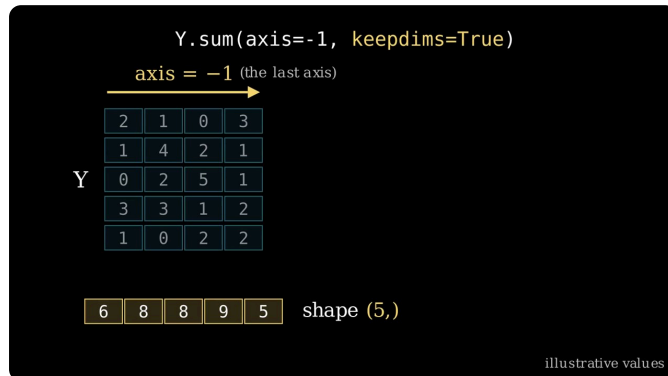
```
import numpy as np
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])
print(A * B)
print(A @ B)
```

CONCEPT 4 OF 7

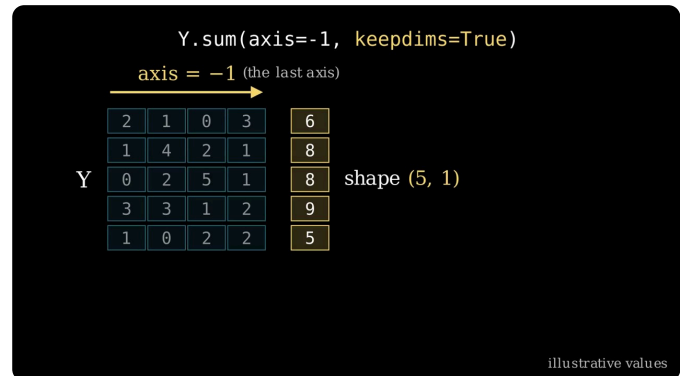
Summing along an axis

▶ Video 0:50-1:01

Many functions take an **axis**: the dimension to work along. `axis=-1` means the **last** axis, which in a matrix runs along each row. So `Y.sum(axis=-1)` gives **one total per row**: 6, 8, 8, 9, 5, with shape (5,). The summed axis disappears. With `keepdims=True` it stays, with size 1: shape **(5, 1)**, a column.



▶ 0:58 One total per row: shape (5,).



▶ 1:01 `keepdims=True`: a column, (5, 1).

KEY IDEA

Summing along an axis removes it: (5, 4) → (5,). `keepdims=True` keeps it with size 1: (5, 4) → (5, 1). Same numbers, different shape.

Check 4: Summing along an axis

Answer all 3 questions before looking at the answers. Pass mark: all correct.

4.1 CALCULATION

For the Y in the picture, work out `Y.sum(axis=0)`, which sums along the first axis: down each column. What is its shape?

4.2 MULTIPLE CHOICE

Which call gives the row totals of Y as a column, shape (5, 1)?

- A. `Y.sum(axis=0)`
- B. `Y.sum(axis=-1)`
- C. `Y.sum(axis=-1, keepdims=True)`
- D. `Y.sum(axis=0, keepdims=True)`

4.3 HANDS-ON CODE

Other functions take an axis too. Predict the three lines.

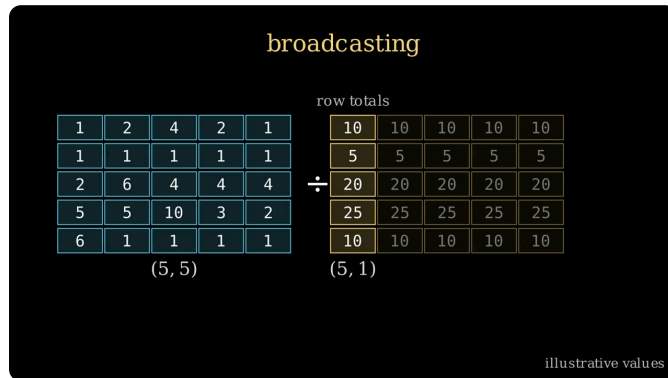
```
import numpy as np
Y = np.array([[2, 1, 0, 3], [1, 4, 2, 1], [0, 2, 5, 1],
              [3, 3, 1, 2], [1, 0, 2, 2]])
print(Y.max(axis=-1))
print(Y.mean(axis=-1, keepdims=True).shape)
print(np.ones((12, 5, 64)).sum(axis=-1).shape)
```

CONCEPT 5 OF 7

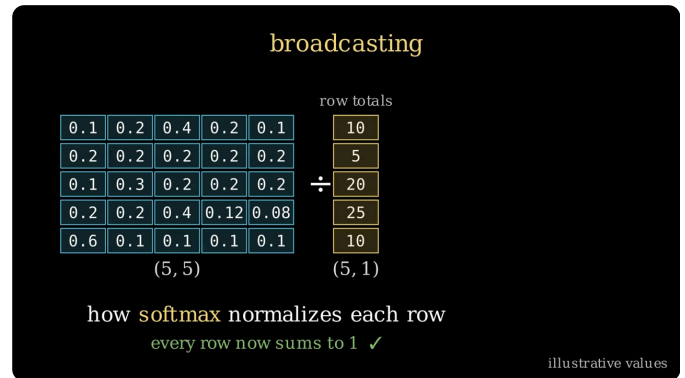
Broadcasting

▶ Video 1:02-1:15

That column lines up with the rows because of **broadcasting**: when an array has size 1 along an axis, NumPy stretches it to match the other array, as if it were copied (the faint columns in the picture). Divide a (5, 5) array by a (5, 1) column, and **each row is divided by its own number**. That is exactly how **softmax** normalizes every row (F07): afterwards, every row adds up to 1.



▶ 1:09 The (5, 1) column is stretched ...



▶ 1:15 ... so each row is divided by its total.

KEY IDEA

A size-1 axis is stretched to match: $(5, 5) / (5, 1) \rightarrow (5, 5)$, each row divided by its own total. A single number is stretched to every element.

Check 5: Broadcasting

Answer all 4 questions before looking at the answers. Pass mark: all correct.

5.1 CALCULATION

Row 4 of E is [5, 5, 10, 3, 2]. Find its total, divide the row by it, and check that the result adds up to 1.

5.2 MULTIPLE CHOICE

E has shape (5, 5). Which line raises an error?

- A. `E / E.sum(axis=-1, keepdims=True)`
- B. `E + 1`
- C. `E * np.ones((1, 5))`
- D. `E + np.ones((5, 2))`

5.3 SHORT ANSWER

Write it yourself. S is a (5, 5) array of scores. In two lines, using `np.exp`, a sum with axis and keepdims, and a division, compute P, the softmax of every row.

5.4 HANDS-ON CODE

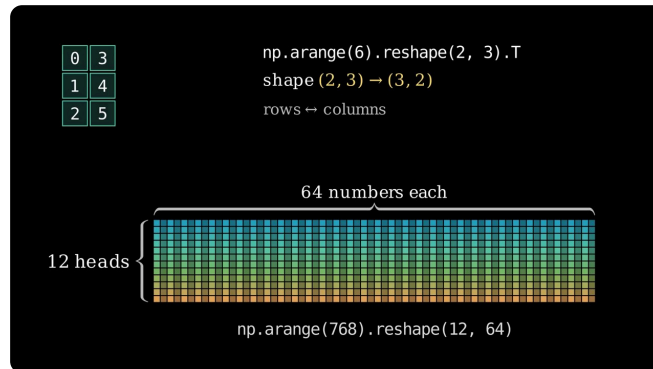
Try it yourself. The second division forgets `keepdims=True`. Predict the three lines. Why does `bad` run without an error, and what did it divide by?

```
import numpy as np
E = np.array([[1, 2, 4, 2, 1], [1, 1, 1, 1, 1], [2, 6, 4, 4, 4],
              [5, 5, 10, 3, 2], [6, 1, 1, 1, 1]])
good = E / E.sum(axis=-1, keepdims=True)
bad  = E / E.sum(axis=-1)
print(E.sum(axis=-1).shape)
print(good.sum(axis=-1))
print(bad[0])
```

Reshape and transpose

▶ Video 1:16-1:25

`reshape` regroups the **same numbers, in the same order**, into a new shape: `np.arange(6)` is 0 1 2 3 4 5, and `.reshape(2, 3)` fills 2 rows of 3, `[[0 1 2], [3 4 5]]`. The total must not change: 768 numbers can become **12 heads of 64** because $12 \times 64 = 768$ (episode 7). `.T` **transposes**: it swaps the axes, so rows become columns and (2, 3) becomes (3, 2).



▶ 1:25 768 numbers as 12 heads of 64; top left, a transpose.

KEY IDEA

`reshape` changes the grouping, never the numbers or their order, so the sizes must multiply to the same total. `.T` swaps rows and columns: $(n, m) \rightarrow (m, n)$.

Check 6: Reshape and transpose

Answer all 4 questions before looking at the answers. Pass mark: all correct.

6.1 MULTIPLE CHOICE

Which of these raises an error for `np.arange(768)` ?

- A. `.reshape(12, 64)`
- B. `.reshape(8, 96)`
- C. `.reshape(10, 76)`
- D. `.reshape(3, 256)`

6.2 HANDS-ON CODE

Both results have shape (3, 2). Predict them. Are they the same array?

```
import numpy as np
a = np.arange(6)
print(a.reshape(3, 2))
print(a.reshape(2, 3).T)
```

6.3 CALCULATION

`x` has shape (5, 64). What is the shape of `x.T`? And of `x @ x.T`?

6.4 TRUE OR FALSE

“Reshaping 768 numbers into (12, 64) changes some of the numbers.”

True ☐ False ☐

CONCEPT 7 OF 7

Masks, and it all together

▶ Video 1:26-1:46

`np.triu` builds an **upper triangle**: `np.triu(np.ones((5, 5)), k=1)` has 1s above the diagonal and 0s elsewhere, and `.astype(bool)` turns them into True/False: a **mask**. `scores[mask] = -np.inf` picks out exactly the True cells and sets them to $-\infty$. Row *i* is a token, column *j* a token it could look at, and the True cells are the **later** tokens. That is the **causal mask** of episode 6: after softmax, $e^{-\infty} = 0$, so no token sees the future.

the causal mask (episode 6)

▶ 1:39 The causal mask: $-\infty$ above the diagonal.

Ep 6
Ep 6-10
Ep 7
Ep 6

▶ 1:45 The toolkit in eight lines.

KEY IDEA

A True/False **mask** picks cells: `scores[mask] = -np.inf` changes only the True ones. `np.triu(..., k=1)` marks everything above the diagonal: the future.

Check 7: Masks, and it all together

Answer all 4 questions before looking at the answers. Pass mark: all correct.

7.1 CALCULATION

How many cells are set to $-\infty$ by the causal mask for 5 tokens? For 10 tokens?

7.2 MULTIPLE CHOICE

In the picture, which tokens can *sat* (the third row) still look at?

- A. Only *sat*
- B. *The*, *cat* and *sat*
- C. *sat*, *on* and *the*
- D. All five

7.3 TRUE OR FALSE

“`np.triu(np.ones((5, 5)), k=1)` already contains $-\infty$ values.”

True ☐ False ☐

7.4 HANDS-ON CODE

Try it yourself. This is the code from the video, with two prints added. (a) What does `mask.sum()` print? (b) What does `scores[1]` print? (c) With `k=0` instead of `k=1`, how many cells would be masked, and what would go wrong for the first token, *The*?

```
import numpy as np
X = np.random.randn(5, 4)           # (5, 4)
W = np.random.randn(4, 4)
Y = X @ W                           # (5, 4)
totals = Y.sum(axis=-1, keepdims=True) # (5, 1)
heads = np.arange(768).reshape(12, 64) # (12, 64)
mask = np.triu(np.ones((5, 5)), k=1).astype(bool)
scores = np.zeros((5, 5))
scores[mask] = -np.inf               # hide future

print(mask.sum())                    # (a)
print(scores[1])                     # (b)
```

ANSWERS

Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

1. Arrays and shapes · rewatch ▶ 0:08-0:29

1.1 B. Rows first, then the numbers in each row. (6,) would be the same 6 numbers as one flat vector.

1.2 (7, 768), 5,376 numbers. One row per token, 768 numbers per row: $7 \times 768 = 5,376$.

1.3 (4,) (5, 4) (4,) v is 1-D with 4 numbers. `X[0]` is one row, one token's vector of 4 numbers, so it is 1-D too: (4,).

2. Arithmetic on every element at once · rewatch ▶ 0:29-0:39

2.1 [[9 18 27] [36 45 54]] · [[3 5 7] [9 11 13]] · [[10 40 90] [160 250 360]] Each position on its own: $10 - 1 = 9$, $20 - 2 = 18$, ...; $1 \times 2 + 1 = 3$, $2 \times 2 + 1 = 5$, ...; and `*` multiplies matching elements: 1×10 , 2×20 , ..., $6 \times 60 = 360$.

2.2 False. Element-wise needs the elements to match up. NumPy raises an error: "operands could not be broadcast together".

2.3 A. `**` squares each element. `x @ x` is the dot product 14 (next concept), `x * 2` is [2, 4, 6], and D squares the total: 36.

3. The @ sign: matrix multiplication · rewatch ▶ 0:40-0:49

3.1 (a) (5, 8) · (b) (5, 5) · (c) an error. Rows from the left, columns from the right. (b) gives one dot product for every pair of the 5 rows, like the scores `Q @ K.T` in episode 6. (c) Rows of 5 numbers can't be dotted with columns of 4: the inner sizes don't match.

3.2 True. The result is (5, 4), so 20 cells. Each is one row (4 numbers) dotted with one column (4 numbers).

3.3 [[5 12] [21 32]] · [[19 22] [43 50]] `*` multiplies matching elements (1×5 , 2×6 , ...). `@` takes dot products: row 1 · column 1 = $1 \times 5 + 2 \times 7 = 19$, row 1 · column 2 = $1 \times 6 + 2 \times 8 = 22$, row 2 · column 1 = $3 \times 5 + 4 \times 7 = 43$, row 2 · column 2 = $3 \times 6 + 4 \times 8 = 50$.

4. Summing along an axis · rewatch ▶ 0:50-1:01

4.1 [7 10 10 9], shape (4,). Column by column: $2 + 1 + 0 + 3 + 1 = 7$, $1 + 4 + 2 + 3 + 0 = 10$, $0 + 2 + 5 + 1 + 2 = 10$, $3 + 1 + 1 + 2 + 2 = 9$.

4.2 C. A gives the column totals (4,), B the same row totals as C but flat, (5,), and D the column totals kept as a row, (1, 4).

4.3 [3 4 5 3 2] · (5, 1) · (12, 5) The largest number in each row; the row means kept as a column; and for 3-D arrays the last axis (64) disappears, leaving (12, 5).

5. Broadcasting · rewatch ► 1:02-1:15

5.1 Total 25 → [0.2, 0.2, 0.4, 0.12, 0.08]. $5/25 = 0.2$, $10/25 = 0.4$, $3/25 = 0.12$, $2/25 = 0.08$, and $0.2 + 0.2 + 0.4 + 0.12 + 0.08 = 1$, as in the picture.

5.2 D. A size of 2 can't be stretched to 5; only size 1 can. A stretches a column, B a single number, C a row (1, 5).

5.3 `e = np.exp(S)` and `P = e / e.sum(axis=-1, keepdims=True)`. Exponentiate every score, then divide each row by its own total (a (5, 1) column, broadcast). `P.sum(axis=-1)` is then all 1s.

5.4 (5,) · [1. 1. 1. 1. 1.] · [0.1 0.4 0.2 0.08 0.1] A (5,) array is lined up with the *last* axis, like a row (1, 5), so it is stretched down the columns: column j is divided by the total of row j . Row 0 becomes $1/10$, $2/5$, $4/20$, $2/25$, $1/10$, which adds up to 0.88, not 1. No error, just a wrong answer: this is why the series always writes `keepdims=True`.

6. Reshape and transpose · rewatch ► 1:16-1:25

6.1 C. $10 \times 76 = 760$, not 768. The others all multiply to 768.

6.2 `[[0 1] [2 3] [4 5]] · [[0 3] [1 4] [2 5]]`. No. Reshape fills the new rows in order: 0 1, then 2 3, then 4 5. Transposing `[[0 1 2], [3 4 5]]` turns its columns into rows. Same shape, different arrangement.

6.3 (64, 5) and (5, 5). $(5, 64) @ (64, 5) \rightarrow (5, 5)$: a dot product for every pair of rows. This is how episode 6 computes `Q @ K.T`.

6.4 False. The same 768 numbers stay in the same order; they are only grouped into 12 rows of 64.

7. Masks, and it all together · rewatch ► 1:26-1:46

7.1 10 and 45. The cells above the diagonal: $4 + 3 + 2 + 1 + 0 = 10$ for 5 tokens, and $9 + 8 + \dots + 1 = 45$ for 10.

7.2 B. Its row has 0 in the first three columns and $-\infty$ for *on* and *the*, which come later. A token sees itself and everything before it.

7.3 False. It holds only 1s and 0s. The $-\infty$ comes from `scores[mask] = -np.inf`.

7.4 (a) 10 · (b) [0. 0. -inf -inf -inf] · (c) 15 (a) True counts as 1, so the sum counts the masked cells. (b) Row 1 is *cat*: it may look at *The* and itself, and the three later tokens are $-\infty$. (c) `k=0` includes the diagonal: $10 + 5 = 15$ cells, so every token is hidden from itself, and *The* has nothing left to look at: its whole row is $-\infty$.